Estruturas Genéricas

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

9 de junho de 2025

Roteiro

Conceitos Gerais
- Objetivos

Listas
- Conceitos Gerais
- Operações

Roteiro

Estrutura de Dados: Pilha e Fila
- Características Gerais

Pilha (Stack)
- Conceitos Gerais

Roteiro

Fila (Queue)

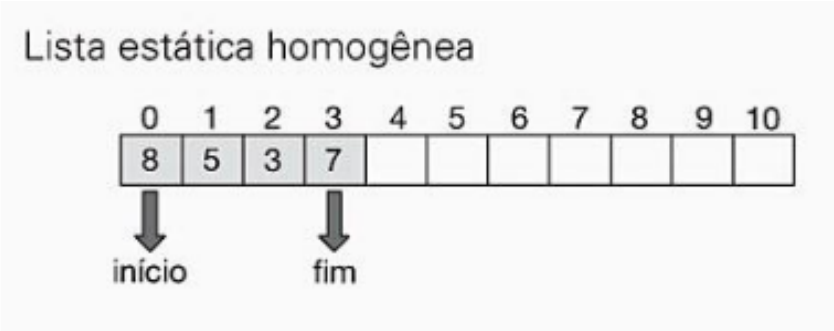
Conceitos Gerais

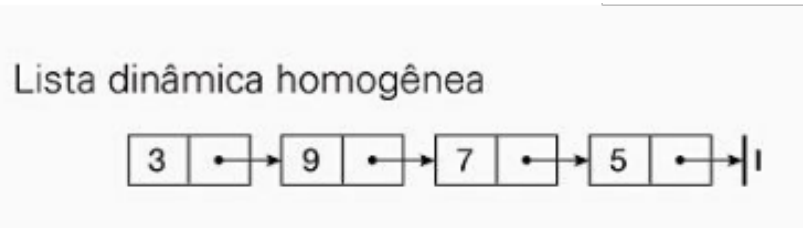
Trabalho 1 - Estruturas Genéricas.

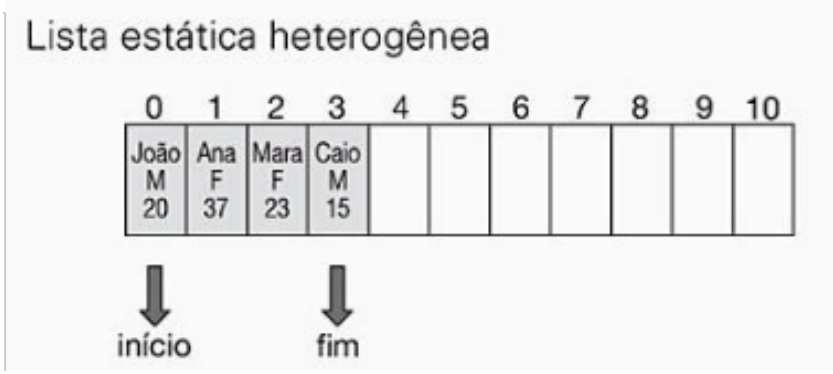
- Esta aula apresenta conceitos gerais pilhas, filas e listas. Ao final, você deverá compreender os seguintes tópicos:
  - Representação visual dessas estruturas
  - Representação computacional dessas estruturas
  - Algoritmos de Pesquisa, Remoção e Inserção
  - Aplicações Reais

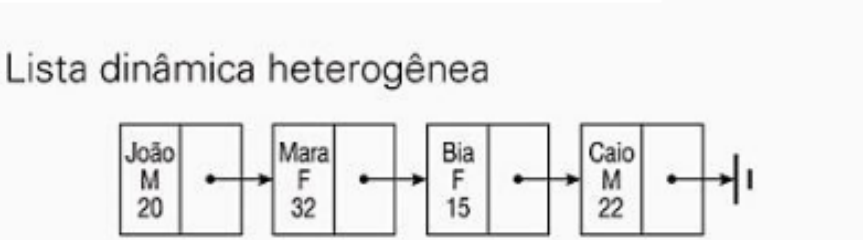
- É uma estrutura de dados na qual os objetos estão organizados em ordem linear.
- Listas Estáticas
  - representadas por vetores e índices.
- Listas dinâmicas
  - representadas por ponteiro (aponta para próximo elemento). Por isso o nome "Lista encadeada"
  - Toda lista encadeada tem pelo menos um ponteiro para o início.

- A lista possui apenas um dado primitivo é dita **homogênea**
- A lista possui dados compostos é dita **heterogênea**









- Podemos implementar as listas de forma estática e dinâmica. Existem 5 tipos:

Lista simplesmente encadeada e não ordenada

Lista simplesmente encadeada e ordenada

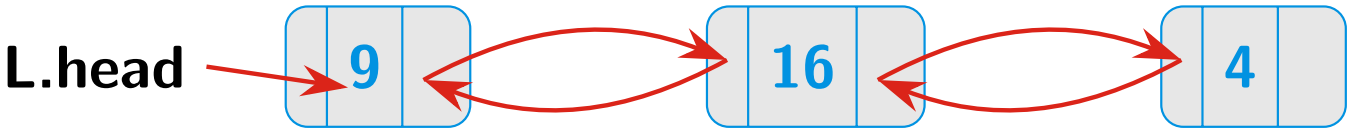
Lista duplamente encadeada e não ordenada

Lista duplamente encadeada e ordenada

Lista circular

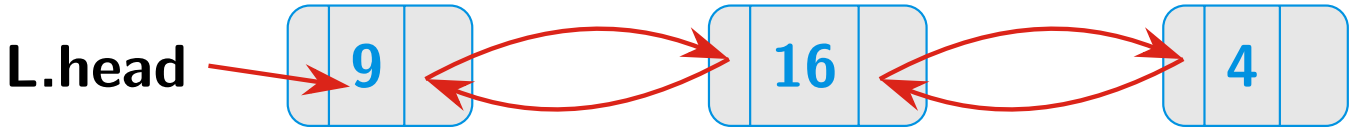
Estrutura Geral - Lista dinâmica

- A ordem em uma lista encadeada é determinada por um ponteiro em cada objeto.
- Suporta diversas operações.
- **Lista duplamente encadeada**: Cada elemento possui uma chave e ponteiros para o nó anterior e o próximo.



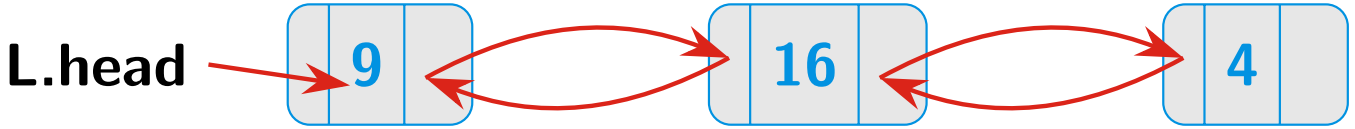
Estrutura Geral - Lista dinâmica

- Cada elemento da lista possui 3 atributos
  - **chave**
  - **ponteiro** para **próximo** elemento (sucessor)
  - **ponteiro** para elemento **anterior** (predecessor)



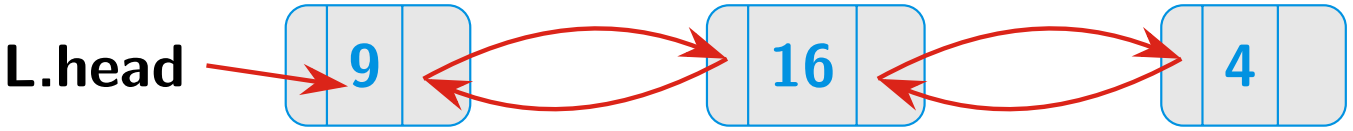
Estrutura Geral - Lista dinâmica

- Dado x um elemento da lista
  - Se $x.anterior = null$ então é o primeiro elemento da lista
  - Se $x.proximo = null$ então é o último elemento da lista
- $L.head$ ou $L.inicio$ que aponta para primeiro elemento da lista
  - $L.inicio = null$ a lista é vazia



Estrutura Geral - Lista dinâmica

- Caso queira implementar uma lista simplesmente encadeada. Que devemos alterar ?
- Caso queira implementar uma lista circular. Que devemos alterar ?
- Caso queira implementar uma lista ordenada. Que devemos alterar ?



- Busca
- Inserção
- Remoção

Implementação em C

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int chave;
    struct node *anterior;
    struct node *proximo;
};

struct node *head = NULL;//Nosso L.inicio ou L.head
```

Implementação em Python

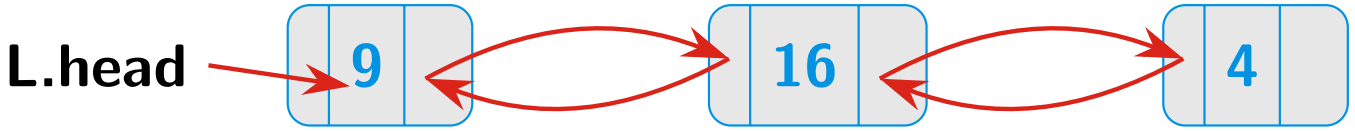
```
1  """
2  class Nodo:
3      def __init__(self, valor):
4          self.chave = valor
5          self.proximo = None #proximo
6          self.anterior = None #anterior
7
8
9  class ListaDuplamenteEncadeada:
10     def __init__(self):
11         self.inicio = None;
```

Operações: Busca

- O procedimento $List_Search(L, K)$ realiza uma busca linear em busca da chave k retornando um ponteiro para esse elemento
- Se nenhum objeto for encontrado, busca retorna *Null*

Operações: Busca

- O procedimento $List_Search(L, K)$ realiza uma busca linear em busca da chave k retornando um ponteiro para esse elemento
- Se nenhum objeto for encontrado, busca retorna *Null*



- $List_Search(L, 4)$ Retorna qual elemento ?
- $List_Search(L, 88)$ Retorna qual elemento ?
- Se existir dois elementos repetidos ? Qual retornar?

Algoritmo: Busca em Lista

Busca Linear (LIST-SEARCH)

```
procedure LIST-SEARCH(L, k)
    x ← L.inicio                    ▷ Inicia a busca pelo primeiro nó
    while x ≠ NIL and x.chave ≠ k do
        x ← x.proximo               ▷ Avança para o próximo nó
    end while
    return x                        ▷ Retorna o nó ou NIL se não encontrar
end procedure
```

- A busca percorre a lista sequencialmente a partir do início.
- **Complexidade (Pior Caso):** $O(n)$, pois pode ser necessário percorrer todos os n elementos.

Listas: Análise e Aplicações

Análise de Complexidade (Lista Duplamente Encadeada com n elementos)

Operação	Melhor Caso	Médio	Pior Caso
Acesso / Busca	$O(1)$	$O(n)$	$O(n)$
Inserção (início)	$O(1)$	$O(1)$	$O(1)$
Inserção (final)	$O(1)$	$O(1)$	$O(1)$
Remoção (início)	$O(1)$	$O(1)$	$O(1)$
Remoção (meio)	$O(1)$	$O(n)$	$O(n)$

Vantagens

- Tamanho dinâmico, cresce conforme a necessidade.
- Inserção e remoção eficientes nas extremidades ($O(1)$).

Desvantagens

- Acesso lento a elementos (requer travessia).
- Maior consumo de memória devido aos ponteiros.

Implementação em C

```
int list_search(int k)
{
    struct node *x = head;//x = L.inicio
    int indice = 0;
    while(x != NULL) {
        if(x->chave == k)
          return indice;
        indice = indice + 1;
        x = x->proximo;
    }
    return -1;
}
```

Implementação em Python

```
1  ""     #Codigo Anterior
2
3      def list_search(self,k):
4          x = self.inicio;
5          while x != None and x.chave != k:
6              x = x.proximo;
7          return x;
```

Análise

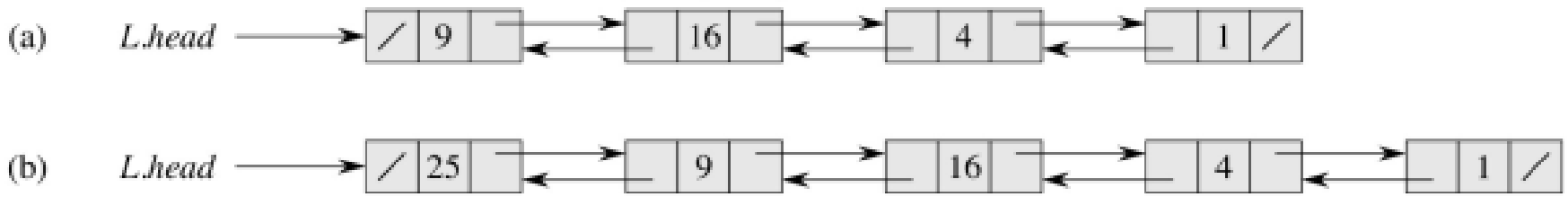
Análise Assintótica

- Considerando lista de tamanho n

  Pior Caso $O(n)$

Operações: Inserção

- O procedimento $List_Insert(L, x)$ insere um elemento **x** no início da lista



Algoritmo: Inserção

```
List-Insert(x)
    if L.inicio == Null
        L.inicio = x
        End Processo


    x.proximo = L.inicio
    if L.inicio != Null
        L.inicio.anterior = x
    L.inicio = x
    x.anterior = Null
```

Implementação em C

```
void list_insert(int value) {
    //Criacao do Nodo
    struct node *x = (struct node*) malloc(sizeof(struct
        node));
    x->chave = value;
    x->anterior = NULL;
    x->proximo = NULL;

    if(head == NULL) {
        head = x;//L.inicio = x
    } else {
        x->proximo = head;
        if(head != NULL)
            head->anterior = x;
        head = x;
        x->anterior = NULL;
    }
}
```

Implementação em Python

```
""     #Codigo Anterior

    def list_insert(self,data):
        x = Nodo(data);
        # Se lista vazia entao inserir no Inicio
        if self.inicio is None:
            self.inicio = x
            return;

        x.proximo = self.inicio;
        if self.inicio != None:
            self.inicio.anterior = x;
        self.inicio = x
        x.anterior = None
```

Implementação em C

```
void print_list() {
    struct node *temp = head;
    while(temp != NULL) {
        printf("%d ", temp->chave);
        temp = temp->proximo;
    }
}
```

Implementação em Python

```
1  "
2      def print_list(self):
3          current = self.inicio
4          while current:
5              print(current.chave)
6              current = current.proximo
```

Análise

Análise Assintótica

- O tempo de execução para $List_Insert(L, x)$ inserir um elemento na lista de tamanho n.

  Pior Caso $O(1)$

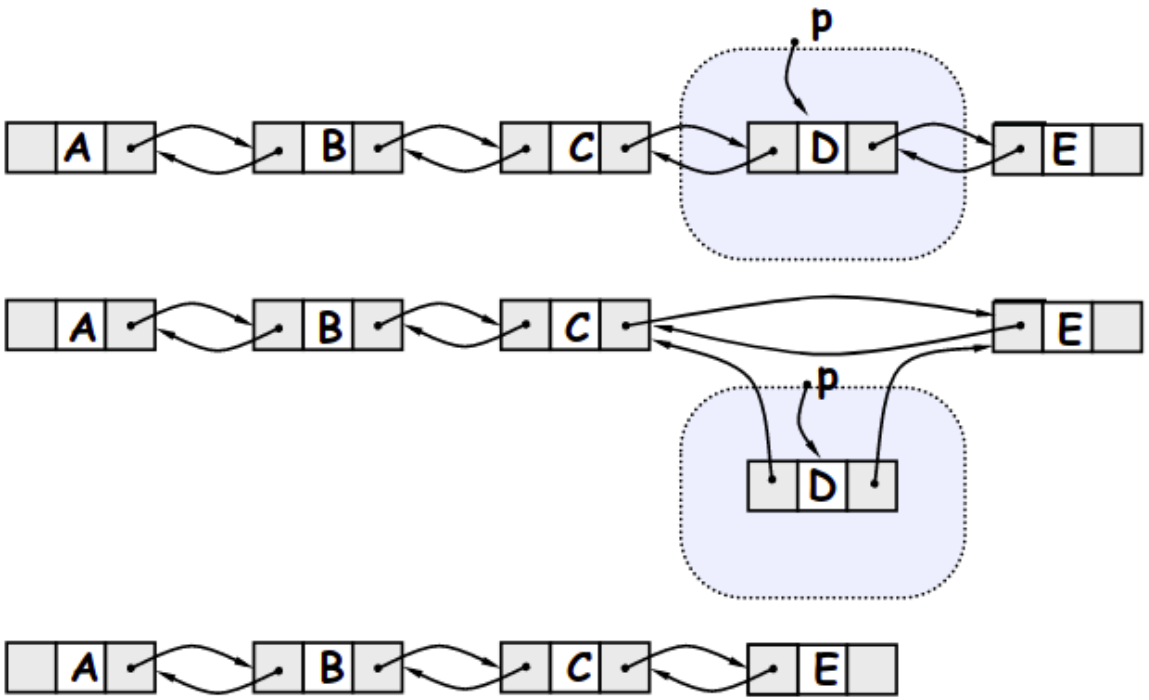
- Qual o tempo de execução para uma lista de tamanho n, caso a inserção fosse no final da lista ?

Operações: Remoção

- O procedimento *List_Delete*(L) remove um elemento **x** no início da lista

Operações: Remoção

- O procedimento *List_Delete*(L) remove um elemento **x** no início da lista



Análise

Análise Assintótica

- O tempo de execução para $List_Delete(L, x)$ remover um elemento na lista de tamanho n.

  Melhor Caso $O(1)$
  Pior Caso $O(n)$

Algoritmo: Remoção

```
LIST-DELETE(L, x)
if x.anterior ≠ NIL
    x.anterior.próximo = x.próximo
else L.início = x.próximo
if x.próximo ≠ NIL
    x.próximo.anterior = x.anterior
```

Algoritmo: Remoção

```
List-delete(x)
    current = L.inicio
    while current != Null
        if current.chave == x
        {
            if current.anterior != Null
                current.anterior.proximo = current.proximo
            else
                L.inicio = current.proximo

            if current.proximo != Null
                current.proximo.anterior = current.anterior

            End Processo
        }
        current = current.proximo;
```

Implementação em C

```
void list_delete(int x)
{
    struct node *current = head;//x = L.inicio
    while(current != NULL) {
        if(current->chave == x)
        {
            if(current->anterior != NULL)
                current->anterior->proximo = current->
                    proximo;
            else head = current->proximo;
            if (current->proximo != NULL)
                current->proximo->anterior = current->
                    anterior;
        }//Fim do IF
      current = current->proximo;
    }//Fim do While
}//Fim do Metodo
```

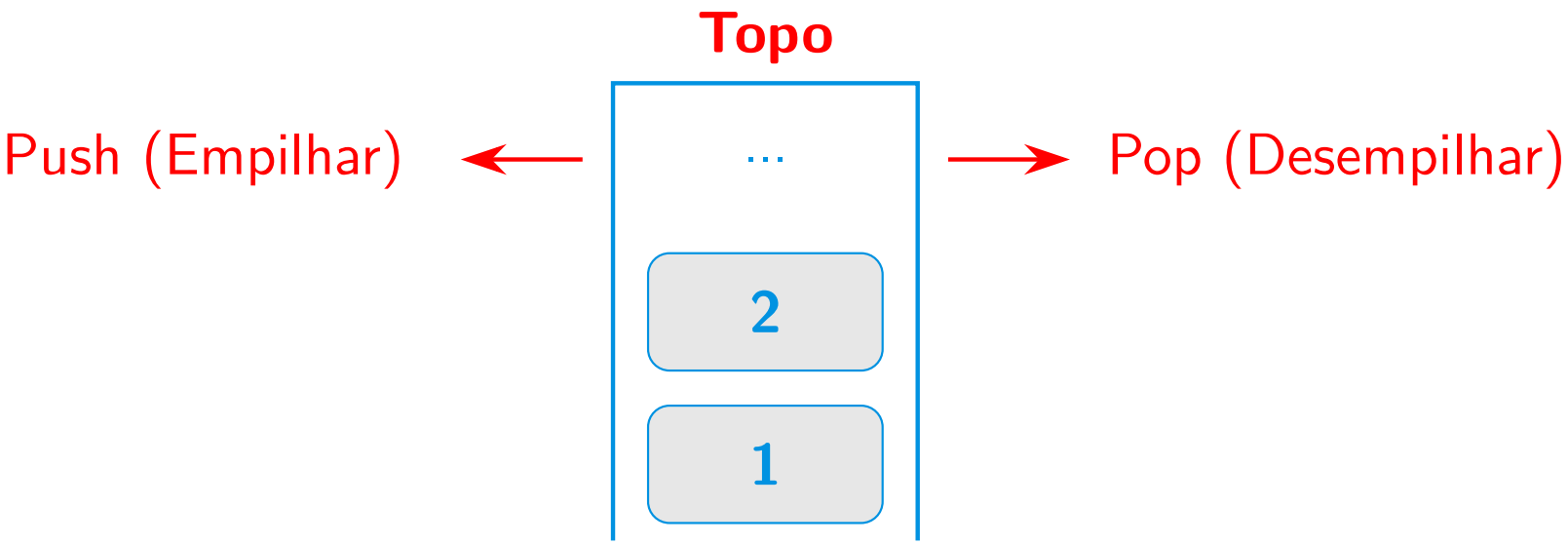
Implementação em Python

```
""     #Codigo Anterior

def list_delete(self,x):
  current = self.inicio;
  while current != None:
    if current.chave == x:
      if current.anterior is not None:
        current.anterior.proximo = current.proximo
      else:
        self.inicio = current.proximo
      if current.proximo is not None:
        current.proximo.anterior = current.anterior
      return
  current = current.proximo;
```

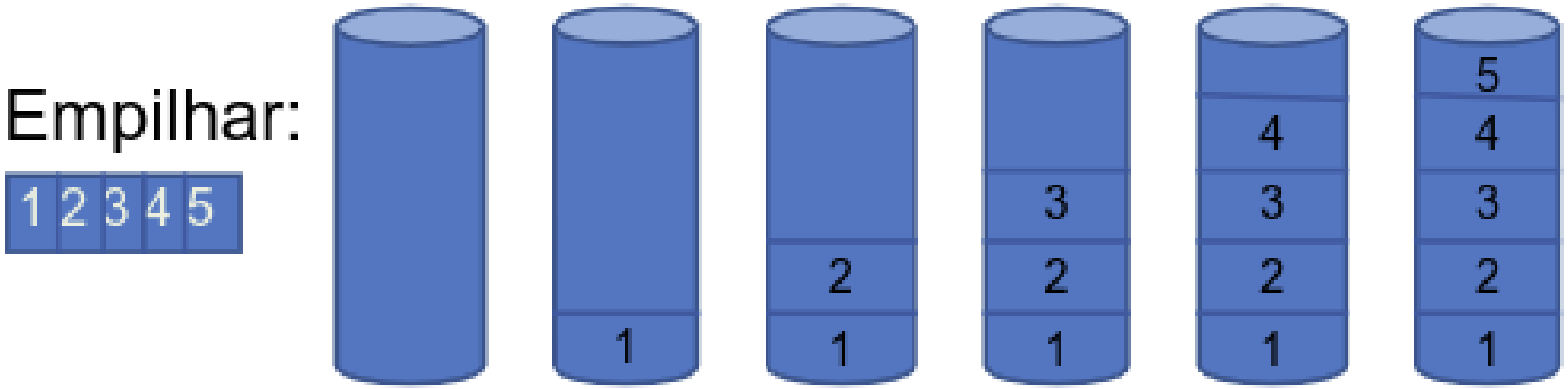
- São consideradas listas especializadas por possuírem características próprias.
- Ambas possuem operações de próprias de inserir, excluir, encontrar o maior elemento e contar os elementos.
- Dados organizados em ordem linear
- Quando representadas por vetores são consideradas **estáticas**. Quando representadas por listas encadeadas com ponteiros, são consideradas **dinâmicas**
- Quando formado por dado primitivo, dizemos que são estruturas **homogênea**. Quando representadas por dados compostos, dizemos que são **heterogênea**.

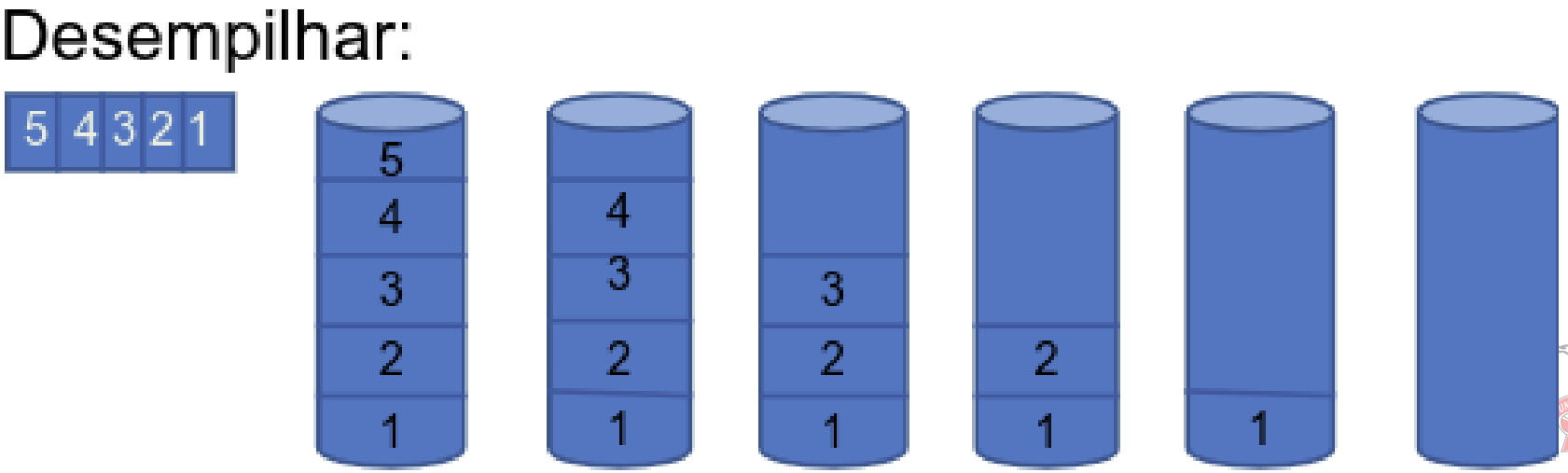
Pilha (Stack): Conceitos Gerais

- Uma coleção ordenada de itens onde as operações ocorrem na mesma extremidade: o **topo**.
- Implementa a política **LIFO (Last-In, First-Out)**: o último a entrar é o primeiro a sair.



Comportamento





Análise

Análise da Complexidade

Push/Pop Operação de tempo constante, então temos $O(1)$

Consulta No pior caso, temos $O(n)$. No melhor caso temos $O(1)$

Algoritmo

```
STACK-EMPTY(S)

if S.topo == 0
    return TRUE
else return FALSE

PUSH(S, x)

S.topo = S.topo + 1
S[S.topo] = x

POP(S)

if STACK-EMPTY(S)
    error "underflow"
else S.topo = S.topo - 1
    return S[S.topo + 1]
```

Implementação em C

```
#include <stdio.h>
#include <stdlib.h>
#include<stdbool.h>
#define MAX_SIZE 10

int stack[MAX_SIZE];
int top = -1;

bool stack_empty()
{
    if(top == 0)
        return true;
    else return false;
}
```

Implementação em C

```
void push(int x) {
    if (top == MAX_SIZE - 1) {printf("Pilha cheia\n");
        return;
    }
    top++;
    stack[top] = x;
}
int pop() {
    if (stack_empty()) {printf("Pilha vazia\n");
        return -1;
    }
    else
    {
        top--;
        return stack[top+1];
    }
}
```

Implementação em Python

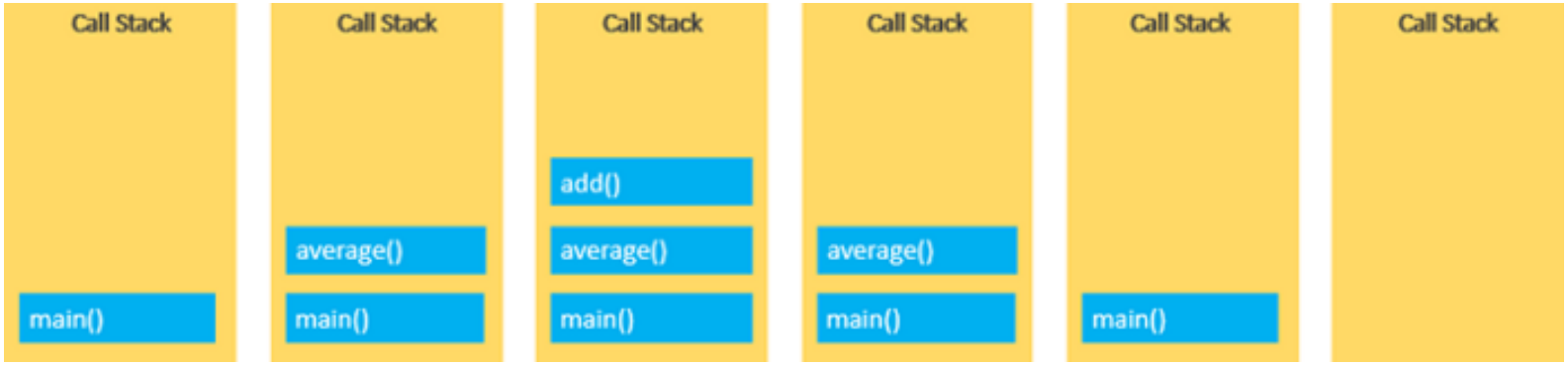
```
""class Stack:
    def __init__(self):
        self.stack = []
        self.top = -1

    def stack_empty(self):
        if self.top == -1:
            return True;
        else:
            return False;
```

Implementação em Python

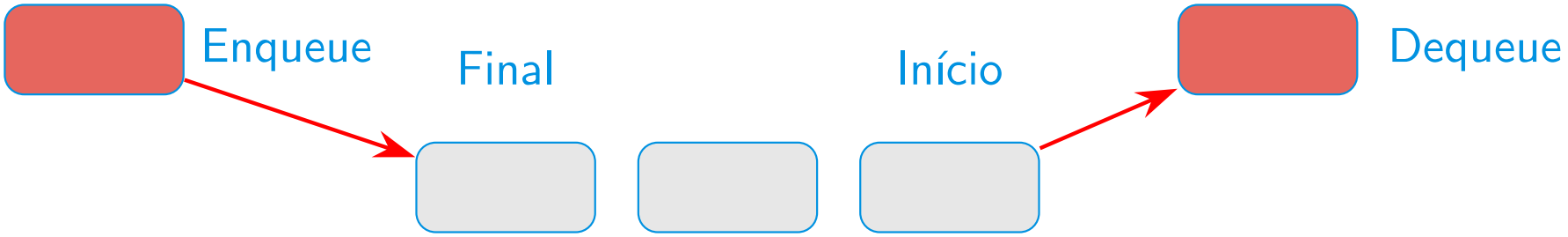
```
""class Stack:

    # Codigo Anterior
    def push(self, item):
        self.top += 1
        self.stack.insert(self.top,item)

    def pop(self):
        if self.stack_empty():
            return None
        item = self.stack[self.top]
        self.top -= 1
        self.stack.pop()
        return item
```

Aplicações

- *Undo/Redo* no editor de textos
- Controle de chamada de funções



- Uma fila é uma coleção ordenada de zero ou mais itens, de um mesmo tipo ou não, tal que suas operações principais, inserção e remoção, são realizadas nas extremidades, denominadas **INICIO (CABEÇA)** e **FIM (CAUDA)** .
- Implementa a política de **primeiro a entrar, primeiro a sair** ou FIFO (*first-in, first-out*).



Operações

Enqueue (Enfileirar) Insere um novo item no **início** da estrutura

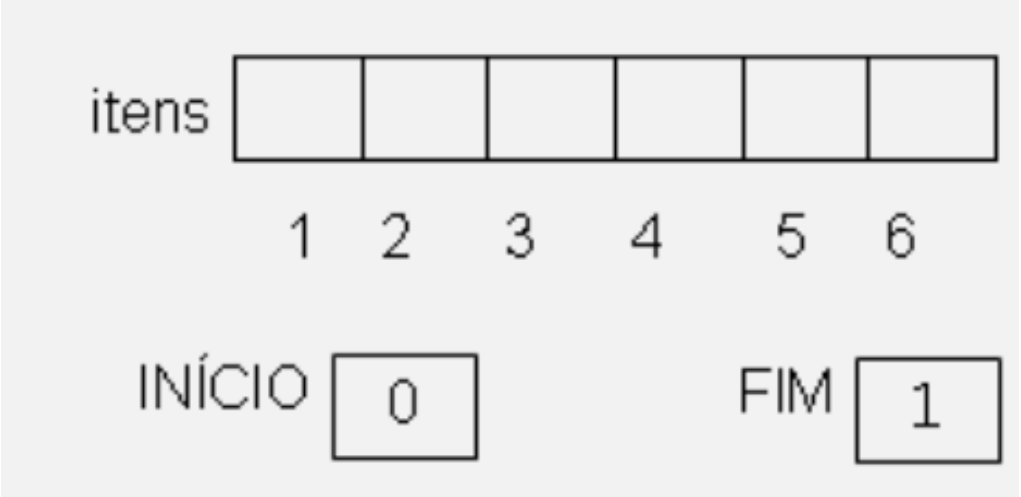
Dequeue (Desinfileirar) Remove um item no **fim** da estrutura

Comportamento

Início aponta para o primeiro da fila, ou seja, o primeiro elemento a sair

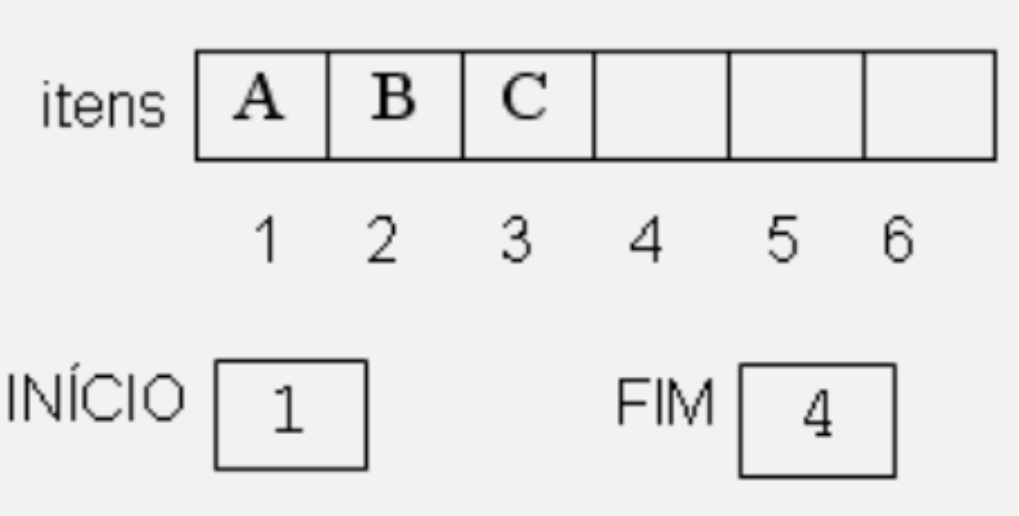
Fim aponta para o fim da fila, ou seja, onde o próximo elemento entrará

Comportamento



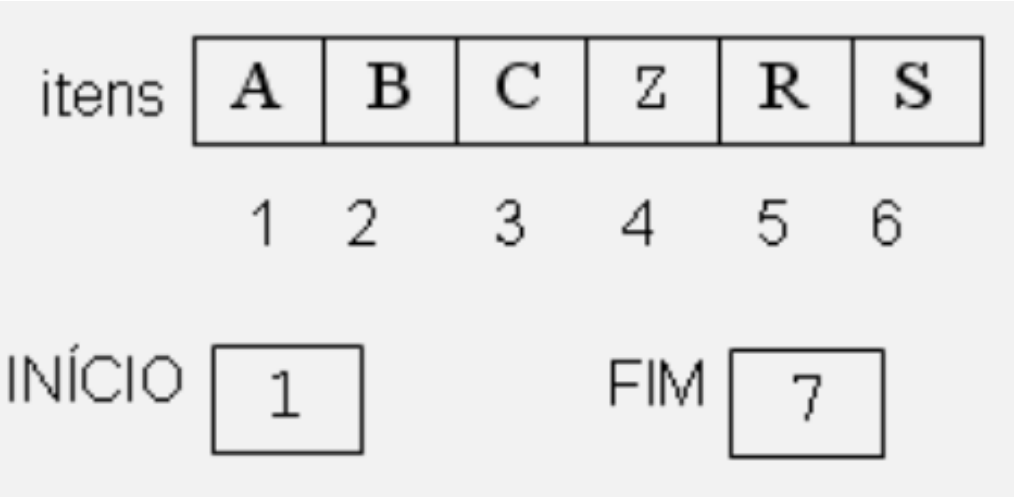
Comportamento

- $Enqueue(A) \rightarrow Enqueue(B) \rightarrow Enqueue(C)$



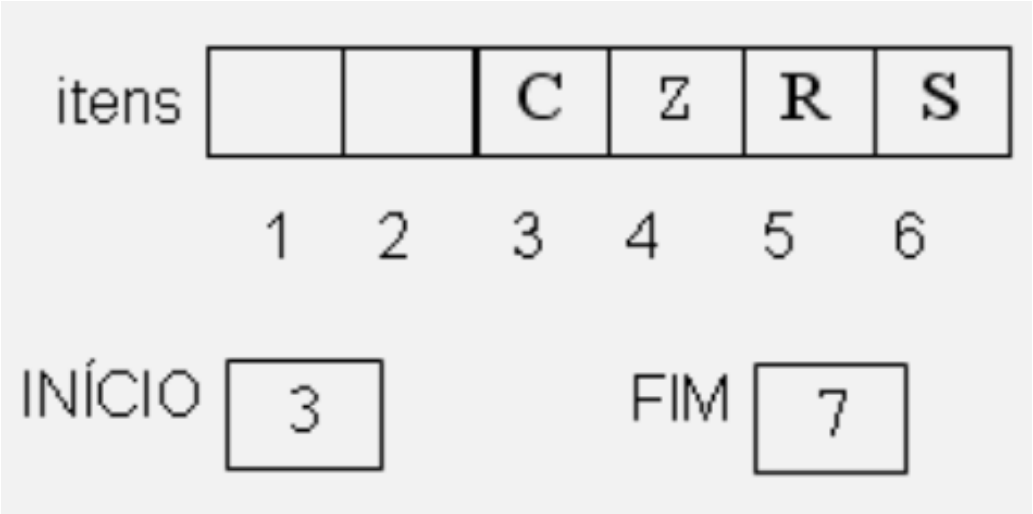
Comportamento

- $Enqueue(Z) \rightarrow Enqueue(R) \rightarrow Enqueue(S)$



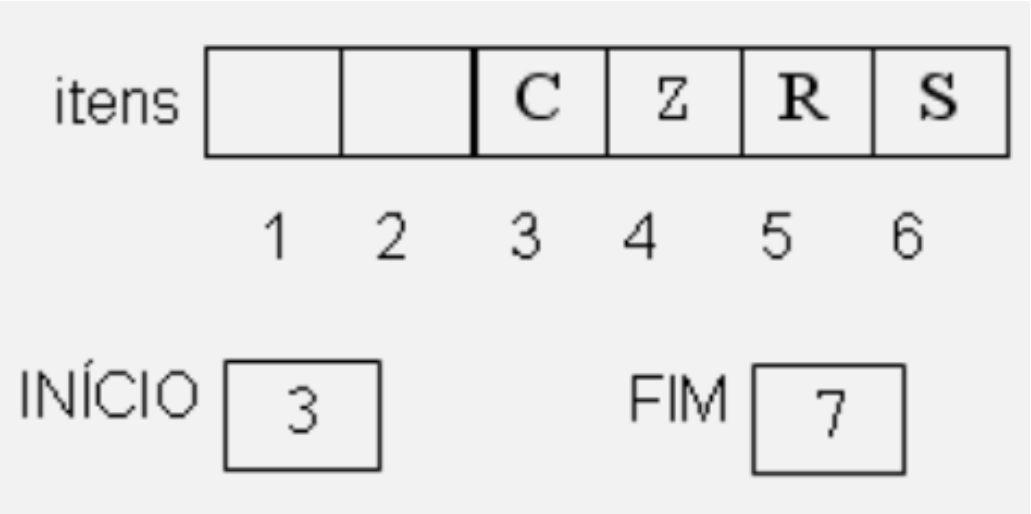
Comportamento

- *Dequeue*() → *Dequeue*()



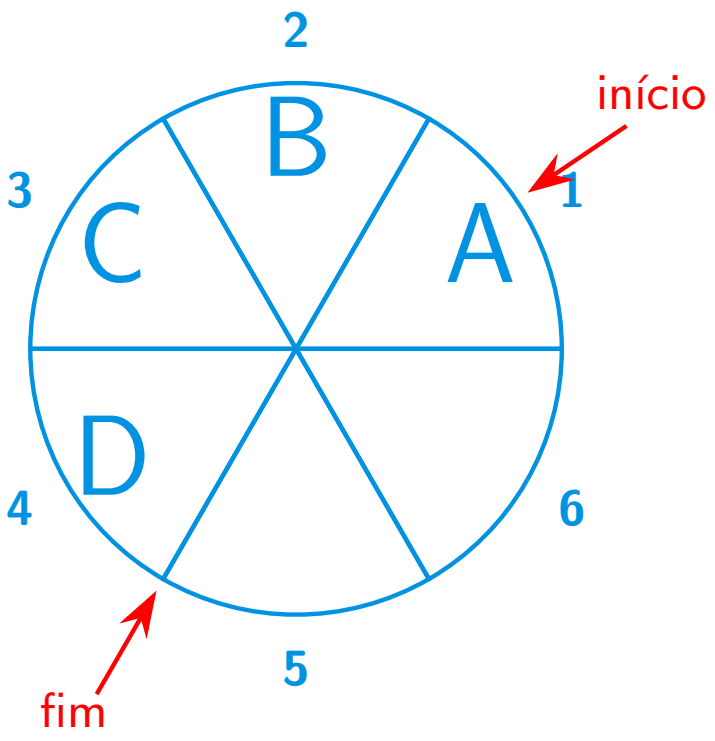
Comportamento

- Temos algumas questões ?
  - Como reutilizar os espaços do início da fila?
  - Como inserir mais elementos?



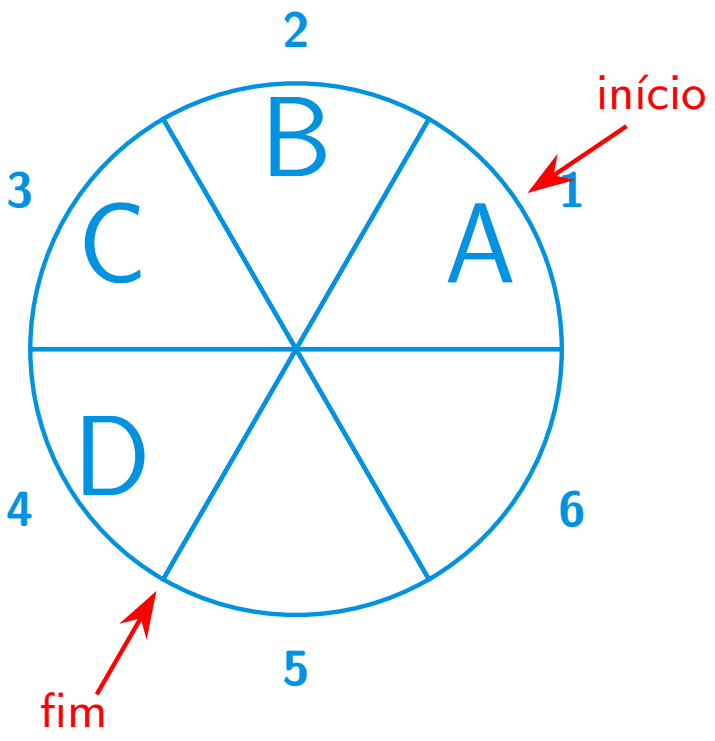
Comportamento: Fila Circular

- A **Fila Circular** otimiza o uso de um vetor, tratando-o como um círculo.
- Permite reutilizar os espaços liberados no início do vetor.
- O cálculo dos índices ‘inicio‘ e ‘fim‘ geralmente usa o operador módulo.



Comportamento

- **Fila em vetor circular**
  - Qual a condição para fila vazia? para fila cheia ? Qual a condição inicial (quando a fila é criada)?



Algoritmo

```
Queue(tamanho_maximo):
    fila = array[tamanho_maximo]
    inicio = 0
    fim = -1

Enqueue(valor):
    se tamanho da fila = tamanho_maximo:
        retornar "Erro: Fila cheia"
    senão:
        fim = fim + 1
        fila[fim] = valor

Dequeue():
    se inicio > fim:
        retornar "Erro: Fila vazia"
    senão:
        valor = fila[inicio]
        inicio = inicio + 1
        retornar valor
```

Algoritmo

```
Front():
    se inicio > fim:
        retornar "Erro: Fila vazia"
    senão:
        retornar fila[inicio]

empty_queue():
    se inicio > fim:
        retornar verdadeiro
    senão:
        retornar falso

queueIsFull():
    se fim == tamanho_maximo - 1:
        retornar verdadeiro
    senão:
        retornar falso
```

Implementação em C

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 10

int queue[MAX_SIZE];
int inicio = 0;
int fim = -1;
int size = 0;//Tamanho
```

Implementação em C

```
void enqueue(int value) {
    if (size == MAX_SIZE) {//Fila Cheia
        return;
    }
    fim = (fim + 1);
    queue[fim] = value;
    size++;
}
int dequeue() {
    if (inicio > fim) {//Fila Vazia
        return -1;
    }
    int value = queue[inicio];
    inicio = (inicio + 1);
    size--;
    return value;
}
```

Implementação em Python.

```
""class Fila:
    def __init__(self, tamanho_maximo):
        self.tamanho_maximo = tamanho_maximo
        self.fila = []
        self.inicio = 0
        self.fim = -1

    def queue_empty(self):
        return self.fim < self.inicio

    def queue_isFull(self):
        return len(self.fila) == self.tamanho_maximo

    def print_front(self):
        if not self.queue_empty():
            print(self.fila[self.inicio])
```

Implementação em Python.

```
""class Fila:

    def enqueue(self, valor):
        if not self.queue_isFull():
            self.fim += 1
            self.fila.insert(self.fim,valor)
    def dequeue(self):
        if not self.queue_empty():
            valor = self.fila[self.inicio]
            self.inicio += 1
            return valor
```

Análise

Análise da Complexidade

Enqueue é uma operação básica, como a de atribuição, e atualização de índices. Logo é uma operação de tempo constante, $O(1)$

Dequeue $O(1)$

Consulta a Fila Como esta operação percorre toda a estrutura de tamanho "n", seu tempo de execução é $O(n)$

Aplicações

- Em computação gráfica, uma aplicação interessante é o algoritmo para colorir regiões gráficas, que consiste em varrer uma matriz de pontos, enfileirar os pontos conectados a esse e trocar a cor, após remover cada um da fila.

1	0	0	2	2
0	2	2	1	2
2	1	1	1	2
2	1	2	1	2
2	2	1	2	2

- Cálculo de distância em Grafos.

- Selecione uma questão e faça:
  1. Implemente uma pilha usando uma lista simplesmente encadeada.
  2. Implemente uma fila usando uma lista simplesmente encadeada.
- Faça a lista usando Ponteiro ou autoreferência (Veja nos Livros).

Referências I

- Lee K.D., Hubbard S. (2015) Trees. In: Data Structures and Algorithms with Python. Undergraduate Topics in Computer Science. Springer, Cham. Retrieved from https://doi.org/10.1007/978-3-319-13072-9_6
- Hubbard, J. (2007). Schaum's Outline sof Data Structures with Java. Retrieved from http://www.amazon.com/Schaums-Outline-Data-Structures-Java/dp/0071476989
- Cormen, T. H., Leiserson, C. E., & Stein, R. L. R. E. C. (2012). Algoritmos: teoria e prática. Retrieved from https://books.google.com.br/books?id=6iA4LgEACAAJ.

Referências II

- Ascenio, Ana Fernanda Gomes. Estrutura de dados: Algoritmos, análise da complexidade e implementações em Java e C++. São Paulo: Pearson Prentice Hall, 2010.
- Szwarcfiter, Jayme Luiz. Estruturas de dados e seus algoritmos / Jayme Luiz Szwarcfiter, Lilian Markenzon. 3.ed. [Reimpr.]. - Rio de Janeiro : LTC, 2015.
- Capítulo 20: Árvores — documentação Aprenda Computação com Python 3.0 2009.1. Retrieved from https://mange.ifrn.edu.br/python/aprenda-com-py3/capitulo_20.html

Estruturas Genéricas

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

9 de junho de 2025